

Reviewer Pack — Minimal Rank of p-Adic Differentials vs AC⁰ Parity Circuit D...

Entry 7897aa7acf7d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 10:23:12 UTC

Minimal Rank of p-Adic Differentials vs AC⁰ Parity Circuit Depth

Entry ID: 7897aa7acf7d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 10:23:12 UTC

1. Conjecture

Field A (mathematical branch): p-Adic Analysis **Field B** (complexity object): Complexity Theory: AC⁰ Parity Circuit Complexity

Statement:

[‘For any polynomial-time computable function $f(n)$, there exists a constant c such that for all $n \leq 40$, the minimal rank of the p-adic differential of a function with an AC⁰ parity circuit depth less than $2^{f(n)}$ is at least $c \cdot \log_2(2^n)$.’, ‘For any instance of size $n \leq 40$ computing the parity function using an AC⁰ circuit of size $2^{f(n)}$, the rank of its corresponding p-adic differential is greater than or equal to a constant multiple of $\log_2(n)$.’]

Rationale (proposer’s reasoning):

[‘p-Adic analysis offers a different perspective on arithmetic operations, which may reveal hidden complexities in functions that are computationally simple. The use of p-adic differential as an invariant could provide new insights into the AC⁰ parity circuit complexity.’, ‘By exploring the relationship between p-adic differentials and circuit complexity, this conjecture might expose previously unknown structures or relationships between these fields.’]

Taxonomy category: AC⁰_PARITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 15dbcf05405608c7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a polynomial-time computable function $f(n)$, if all generated AC⁰ parity circuits with size up to $2^{f(n)}$ have p-adic differentials with a minimal rank greater than or equal to $c \cdot \log_2(2^n)$ for at least 28 out of 30 random seeds, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - p-adic analysis AND AC0 parity circuit depth - minimal rank p-adic differentials AND AC0 complexity - AC0 circuit size $2^{f(n)}$ AND p-adic differential rank $\geq c \cdot \log_2(2^n)$

Top relevant hits considered: - [<http://arxiv.org/abs/math-ph/0512018v2>] On Phase Transitions for P -Adic Potts Model with Competing Interactions on a Cayley Tree - [<http://arxiv.org/abs/2408.00810v3>] p-adic Equiangular Lines and p-adic van Lint-Seidel Relative Bound - [<http://arxiv.org/abs/1012.5548v1>] Supercuspidal characters of SL_2 over a p -adic field

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_parity_function(n):
        return [i % 2 for i in range(1 << n)]

    def ac0_circuit_depth(n):
        if n == 1:
            return 1
        return 1 + ac0_circuit_depth(n - 1)

    def p_adic_differential(f, x):
        diff = []
        for i in range(len(f)):
            diff.append((f[i] - f[(i - 1) % len(f)]) / (x - 1))
        return diff

    def rank(differential):
        m, n = len(differential), len(differential[0])
        if m == 0 or n == 0:
            return 0
        for i in range(m):
```

```

        if differential[i][0] != 0:
            pivot_row = differential[i]
            for j in range(i + 1, m):
                factor = differential[j][0] / pivot_row[0]
                for k in range(n):
                    differential[j][k] -= factor * pivot_row[k]
            rank_value = sum(1 for row in differential if any(row))
            return rank_value

n_min = 5
n_max = 40
instances_tested = 0
total_rank = 0
conjecture_holds = True
counterexample = ""

for n in range(n_min, n_max + 1):
    f = generate_parity_function(n)
    depth = ac0_circuit_depth(n)
    diff = p_adic_differential(f, depth)
    rank_value = rank(diff)

    instances_tested += 1
    total_rank += rank_value

    if rank_value < math.log2(2**n):
        conjecture_holds = False
        counterexample = f"n={n}, rank={rank_value} < log2(2^n)"

mean_rank = total_rank / instances_tested

return {
    "metric_name": "Minimal Rank of p-Adic Differential",
    "metric_value": mean_rank,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction={support_fraction}")

```

```

else:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture"])
    print(f"RESULT: FALSIFIED counterexample='{result['counterexample']}' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_065707ff.py", line 85, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_065707ff.py", line 60, in run_trial
    rank_value = rank(diff)
                  ^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_065707ff.py", line 36, in rank
    m, n = len(differential), len(differential[0])
                               ^^^^^^^^^^^^^^^^^
TypeError: object of type 'float' has no len()

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture according to the pre-registered support and falsification conditions. | next: Investigate the cause of the crash in the test code and attempt to run the test again.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11971
2	propose	ollama_remote	glm4:latest	0	0	11106
3	preregistration	ollama_remote	glm4:latest	0	0	5759
4	novelty	ollama_remote	glm4:latest	0	0	4788
5	novelty	ollama_remote	glm4:latest	0	0	5550
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	45199
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13235
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11096
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8991
10	judge	ollama_remote	glm4:latest	0	0	10762

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 128455 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/7897aa7acf7d.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/7897aa7acf7d.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/7897aa7acf7d.tar.gz (if generated)